Day 1

What Databases Are, Tables, Keys, and Loading the Course Dataset

Phase	Phase 1: Foundations (Days 1-7)
Time	60-90 minutes
You need	Working PostgreSQL + pgAdmin (or SQLite + DB Browser) from Day 0

Learning objectives

- Explain what a database and a relational table are, in plain language.
- Identify rows, columns, primary keys, and foreign keys in a table.
- Describe how the four course tables connect to each other.
- Load the course dataset and confirm it loaded correctly.

Today's plan

Step	Time	What happens
1. Concept	15 min	Databases, tables, rows, columns, keys
2. Guided practice	20 min	Load the dataset, peek at each table
3. Exercises	25-40 min	7 problems, easy to hard
4. Recap	5 min	Takeaway and common mistakes

1. Concept (15 min)

What is a database?

A **database** is an organized collection of data that a computer can store, search, and update quickly and safely. Think of a spreadsheet that has grown up: many linked sheets, millions of rows, many people using it at once, and strict rules so the data stays clean.

A **relational database** (PostgreSQL, SQLite, MySQL, SQL Server) stores data in **tables** that are linked together. **SQL** (Structured Query Language) is the language you use to ask questions of that data. This course teaches you to ask better and better questions over 30 days.

Tables, rows, and columns

- A **table** holds one kind of thing: customers, products, orders.

- A **column** is one attribute of that thing (name, email, city). Each column has a **data type** such as text, whole number, or date.
- A **row** is one individual record (one customer, one order).

Here is a slice of our `customers` table:

customer_id	name	email	city
1	Alice Johnson	alice@example.com	New York
2	Brian Lee	brian@example.com	Chicago
3	Carla Gomez	carla@example.com	Austin

The header row lists the **columns**. Each line below it is a **row**. Three rows, four columns.

NULL: a missing value

Sometimes a value is unknown or does not exist. SQL stores that as **NULL**. NULL is not zero and not an empty string; it means "no value here." One customer in our data has a NULL email on purpose. We will deal with NULL properly on Day 4.

Keys: how rows are identified and linked

- **Primary key (PK):** a column (or set of columns) whose value is unique for every row and never NULL. It is the row's ID card. In `customers`, the PK is `customer_id`.
- **Foreign key (FK):** a column that holds another table's primary key value, creating a link. `orders.customer_id` points to `customers.customer_id`, so every order knows which customer placed it.

Why not store the customer's name inside every order? Because if Alice changes her email, you would have to fix it in many places. Storing it once and linking by key keeps data consistent. This idea is the heart of relational design.

The course dataset: a small online store

Every day of the course uses the same four tables, so you never have to learn a new context.

Table	Primary key	Foreign keys	Meaning
customers	customer_id	-	People who signed up
products	product_id	-	Items for sale
orders	order_id	customer_id → customers	One purchase by one customer
order_items	(order_id, product_id)	order_id → orders product_id → products	Which products are in each order, and how many

The relationships read like sentences: one customer has many orders; one order has many order items; each order item refers to one product. `order_items` uses a **composite** primary key (two columns together), since one order can contain many products but each product appears once per

order.

Heads up: planted quirks
The data has deliberate oddities that later lessons will use: customer 7 has a NULL email, and customers 7 and 8 have placed no orders.

2. Guided Practice (20 min)

Do these in order. Use **pgAdmin's Query Tool** (right-click the `sqlcourse` database, then Query Tool) or the **Execute SQL** tab in DB Browser for SQLite.

Step 1: Load the dataset

Copy the entire script below into the query window and run it once. It works unchanged on PostgreSQL and SQLite. It starts with `DROP TABLE IF EXISTS`, so re-running it safely resets everything.

```
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS customers;

CREATE TABLE customers (
  customer_id  INTEGER PRIMARY KEY,
  name         VARCHAR(50) NOT NULL,
  email        VARCHAR(100),
  city         VARCHAR(50),
  signup_date  DATE
);

CREATE TABLE products (
  product_id  INTEGER PRIMARY KEY,
  name        VARCHAR(50) NOT NULL,
  category    VARCHAR(30),
  price       NUMERIC(8,2)
);

CREATE TABLE orders (
  order_id     INTEGER PRIMARY KEY,
  customer_id  INTEGER NOT NULL REFERENCES customers(customer_id),
  order_date   DATE,
  status       VARCHAR(20)
);

CREATE TABLE order_items (
  order_id    INTEGER NOT NULL REFERENCES orders(order_id),
  product_id  INTEGER NOT NULL REFERENCES products(product_id),
  quantity    INTEGER NOT NULL,
  unit_price  NUMERIC(8,2) NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
```

```
INSERT INTO customers (customer_id, name, email, city, signup_date) VALUES
(1, 'Alice Johnson', 'alice@example.com', 'New York', '2024-01-15'),
(2, 'Brian Lee',     'brian@example.com', 'Chicago',  '2024-02-03'),
(3, 'Carla Gomez',   'carla@example.com', 'Austin',   '2024-02-20'),
(4, 'David Kim',     'david@example.com', 'New York', '2024-03-11'),
(5, 'Emma Wilson',   'emma@example.com',  'Seattle',  '2024-04-05'),
(6, 'Farid Khan',    'farid@example.com', 'Chicago',  '2024-05-19'),
(7, 'Grace Liu',     NULL,                'Austin',   '2024-06-02'),
(8, 'Henry Brown',   'henry@example.com', 'Seattle',  '2024-07-08');

INSERT INTO products (product_id, name, category, price) VALUES
(101, 'Wireless Mouse',      'Electronics', 24.99),
(102, 'Mechanical Keyboard', 'Electronics', 79.00),
(103, 'USB-C Cable',         'Electronics',  9.99),
(104, 'Notebook Set',        'Stationery',  12.50),
(105, 'Gel Pen Pack',        'Stationery',   6.75),
(106, 'Desk Lamp',           'Home',        34.90),
(107, 'Water Bottle',        'Home',        15.00),
(108, 'Backpack',            'Accessories', 49.95);

INSERT INTO orders (order_id, customer_id, order_date, status) VALUES
(1001, 1, '2024-02-01', 'completed'),
(1002, 2, '2024-02-14', 'completed'),
(1003, 1, '2024-03-05', 'completed'),
(1004, 3, '2024-03-22', 'cancelled'),
(1005, 4, '2024-04-10', 'completed'),
(1006, 5, '2024-05-01', 'shipped'),
(1007, 2, '2024-06-15', 'completed'),
(1008, 6, '2024-07-01', 'pending');

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1001, 101, 1, 24.99),
(1001, 103, 2,  9.99),
(1002, 102, 1, 79.00),
(1003, 104, 3, 12.50),
(1003, 105, 2,  6.75),
(1004, 106, 1, 34.90),
(1005, 107, 2, 15.00),
(1005, 103, 1,  9.99),
(1006, 108, 1, 49.95),
(1007, 101, 2, 24.99),
(1007, 105, 1,  6.75),
(1008, 102, 1, 79.00);
```

If you see an error
Run the script from the very top, in one go. On pgAdmin, make sure the Query Tool is connected to **sqlcourse**, not the default **postgres** database. In DB Browser, click **Write Changes** (Ctrl+S) after running so the data is saved.

Step 2: Peek at each table

You have not formally learned `SELECT` yet (that is Day 2), but you need just one pattern today. `SELECT * FROM` means "show me all columns," and `FROM` names the table.

```
SELECT * FROM customers;
```

You should see 8 rows. Look at row 7: the email cell shows NULL (pgAdmin displays *[null]*). Now the rest:

```
SELECT * FROM products;
SELECT * FROM orders;
SELECT * FROM order_items;
```

In pgAdmin, run one statement at a time (highlight it, press F5) to see each result. Expected sizes: products 8 rows, orders 8 rows, order_items 12 rows.

Step 3: Follow a link by eye

Look at order **1003** in `orders`: its `customer_id` is 1. Find customer 1 in `customers`: Alice Johnson. That is a foreign key at work. Next, find the rows for order 1003 in `order_items`: products 104 and 105. Look those up in `products`: Notebook Set and Gel Pen Pack. You have just done by hand what a JOIN will do automatically on Day 13.

3. Exercises (25-40 min)

Use only `SELECT * FROM table_name;` and your eyes. Write your answers down, then check the answer key.

Exercise 1 (Easy)

In your own words, what is the difference between a **row** and a **column**?

Exercise 2 (Easy)

Run `SELECT * FROM products;`. How many rows and how many columns does the result have?

Exercise 3 (Easy)

Name the primary key of each of the four tables.

Exercise 4 (Medium)

Which column in `orders` is a foreign key, and which table and column does it point to?

Exercise 5 (Medium)

Run `SELECT * FROM customers;`. Which customer has a NULL email, and what is that customer's city?

Exercise 6 (Medium)

Using `orders` and `customers` by eye, find the **name** of the customer who placed order 1005.

Exercise 7 (Hard)

By looking at `orders`, which customers (by id) have placed **no** orders? Then explain why `order_items` needs a two-column primary key instead of just `order_id`.

Stretch goal
Without running any queries, predict how many total rows each of these will return: `SELECT * FROM orders;` and `SELECT * FROM order_items;`. Then, as a preview, try `SELECT name, city FROM customers;` and notice that you can choose columns instead of using `*`. Tomorrow's lesson is built on this.

4. Recap (5 min)

One-line takeaway
A relational database stores data in linked tables: rows are records, columns are attributes, primary keys identify rows, and foreign keys connect tables.

Common mistakes

- **Running only part of the load script** or running it in the wrong database, which leaves missing tables. Re-run the whole script in *sqlcourse*.
- **Forgetting the semicolon** (;) at the end of a statement, especially when running several at once.
- **Confusing NULL with an empty value.** NULL means unknown or missing, not blank text and not zero.
- **Mixing up PK and FK.** A PK identifies a row in its own table; an FK points to a row in another table.
- **Forgetting to save** in DB Browser for SQLite (Write Changes).

Answer Key with Explanations

Exercise 1

A **row** is one record (one customer, one order) and runs horizontally. A **column** is one attribute shared by all records (like *city*) and runs vertically. Each row holds one value per column.

Exercise 2

8 rows and 4 columns (product_id, name, category, price). The row count matches the 8 INSERTs; the column count matches the CREATE TABLE.

Exercise 3

customers: `customer_id`; products: `product_id`; orders: `order_id`; order_items: the **combination** (`order_id, product_id`). Each is unique per row and never NULL.

Exercise 4

`orders.customer_id` is a foreign key pointing to `customers.customer_id`. It is what lets each order be tied to the person who placed it.

Exercise 5

Grace Liu (customer 7) has the NULL email, and her city is **Austin**.

Exercise 6

Order 1005 has `customer_id` 4; customer 4 is **David Kim**. This two-step lookup via a foreign key is exactly what a JOIN automates.

Exercise 7

Customers **7 (Grace Liu) and 8 (Henry Brown)** never appear in `orders.customer_id`. The values there are 1, 2, 1, 3, 4, 5, 2, 6. Order_items needs a two-column key because one order can have **several** products, so `order_id` repeats (order 1001 appears twice). Only the pair (order_id, product_id) is unique per row.

Stretch goal answer

`orders` returns **8** rows and `order_items` returns **12**. More items than orders is expected, since orders often contain several products. The preview query returns just two columns (name and city) for all 8 customers.

Coming up: Day 2
You will learn `SELECT` and `FROM` properly: choosing columns, aliases, and `DISTINCT`.